

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E
TECNOLOGIA DE SÃO PAULO**

BRUNO HENRIQUE DA SILVA BALBINO

SISTEMA DE GERENCIAMENTO DE AUTO CENTER

CAMPOS DO JORDÃO

2026

**INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E
TECNOLOGIA DE SÃO PAULO**

PAULO GIOVANI DE FARIA ZEFERINO

DISCIPLINA: BANCO DE DADOS II

**CAMPOS DO JORDÃO
2026**

Resumo

O presente projeto tem como base um sistema de gerenciamento para auto centers, desenvolvido originalmente sobre o modelo relacional (MySQL) durante a disciplina de Banco de Dados 2. A aplicação visa organizar processos fundamentais, como o cadastro de clientes, veículos, ordens de serviço, controle de estoque de peças e agendamentos.

Este relatório apresenta todas as etapas do projeto, abrangendo desde o levantamento do problema, definição de objetivos e fundamentação teórica, incluindo um conjunto de consultas analíticas para a validação do sistema.

A proposta atual visa transferir essa estrutura para o paradigma NoSQL, adotando o MongoDB e a abordagem orientada a documentos como guias da nova modelagem. Esse processo não se resume a uma tradução direta de tabelas para coleções, ele impõe a necessidade de repensar as conexões entre as informações, priorizando a frequência de leitura e o agrupamento estratégico de dados em um mesmo documento, a fim de elevar o desempenho geral do sistema.

Palavras-Chave: Banco de dados; auto center; gerenciamento; sistema; MongoDB .

ABSTRACT

This project is based on a management system for auto repair centers, originally developed using the relational model (MySQL) during the Database 2 course. The application aims to organize key processes, such as the registration of customers, vehicles, and service orders, as well as parts inventory management and appointment scheduling.

This report outlines all project stages, ranging from problem identification, objective definition, and theoretical background to a set of analytical queries for system validation.

The current proposal aims to migrate this structure to the NoSQL paradigm, adopting MongoDB and a document-oriented approach to guide

the new modeling. This process goes beyond a simple translation of tables into collections; it requires rethinking data relationships, prioritizing read frequency and the strategic grouping of data within a single document to enhance overall system performance.

Keywords: Database; Auto Center; Management; system; MongoDB.

SUMÁRIO

1	INTRODUÇÃO	5
1.1	Objetivos	6
1.2	Justificativa	7
1.3	Aspectos Metodológicos	7
1.4	Aporte Teórico	8
2	METODOLOGIA	9
2.1	Considerações Iniciais	9
2.2	O Modelo NoSQL	10
2.3	SGBD Utilizado e o Modelo de Dados	10
3	RESULTADOS OBTIDOS	11
3.1	Criacao das Coleções	12
3.2	Inserção de Dados	12
3.3	Matriz de 30 Consultas NoSQL	17
4	CONCLUSÃO	54
4.1	Resultados Obtidos	54
4.2	Sugestões para Melhorias Futuras	55
5	REFERÊNCIAS	55

1. INTRODUÇÃO

A evolução tecnológica e o aumento da demanda nos centros automotivos nas últimas décadas produziram a necessidade de sistemas capazes de lidar com históricos de serviços complexos, cadastros detalhados e dados heterogêneos sobre veículos e estoques de peças. Nesse cenário, os bancos de dados não relacionais, conhecidos como NoSQL, surgem como alternativa para esses ambientes, nos quais a flexibilidade de esquema para diferentes tipos de registros e o alto desempenho na leitura do histórico dos clientes são fatores prioritários

Este projeto propõe a evolução de um sistema de gerenciamento de centro automotivo , que abrange desde o cadastro de clientes e veículos até o controle de estoque e ordens de serviço do modelo relacional para o paradigma NoSQL. Utilizando o MongoDB como base, a nova modelagem orientada a documentos visa substituir a estrutura tradicional de tabelas por coleções otimizadas. Esse processo exige repensar o relacionamento e o agrupamento estratégico dos dados em documentos únicos, levando em conta as consultas mais frequentes do sistema. O objetivo final é oferecer a pequenas e médias oficinas mecânicas uma solução de alta performance, que simplifique o fluxo de trabalho e promova uma gestão mais ágil e profissional.

1.1 Objetivos

O projeto do sistema para gerenciamento de centro automotivo tem como principal objetivo o desenvolvimento de uma plataforma eficiente para o gerenciamento de serviços, clientes, veículos, ordens de serviço, peças e funcionários, estruturada a partir de um banco de dados NoSQL . O sistema será responsável não apenas pelo controle das operações diárias da oficina, mas também pela organização e integridade dos dados comerciais e operacionais, assegurando segurança, consistência e escalabilidade.

A seguir, apresentam-se os objetivos detalhados com ênfase no banco de dados:

- Estruturar o armazenamento de entidades fundamentais do cenário automotivo (como Clientes, Veículos, Agendamentos, Peças e Ordens de Serviço) utilizando o modelo orientado a documentos (JSON/BSON).
- Desenvolver rotinas avançadas de manipulação de dados utilizando o Aggregation Framework para realizar agrupamentos, substituindo subconsultas e junções complexas do modelo relacional pela leitura otimizada de vetores e documentos aninhados (ou junções pontuais via operador \$lookup).
- Desenvolver consultas otimizadas para facilitar a busca por informações, como histórico de ordens de serviço por veículo, cliente, data, tipo de serviço ou funcionário responsável, garantindo alta performance de leitura.
- Adaptar estruturas de controle gerencial do modelo relacional (como Views e Funções de Usuário - UDFs) para o ecossistema NoSQL, utilizando exibições virtuais nativas do banco e scripts de decisão estruturada desenvolvidos em JavaScript diretamente no shell.
- Controlar o estoque de peças, possibilitando o acompanhamento das saídas e entradas de materiais, evitando rupturas e perdas operacionais.
- Gerenciar o histórico de atendimentos, permitindo o registro detalhado de serviços executados, peças utilizadas e responsáveis técnicos, consolidando essas informações de forma desnormalizada dentro do documento da Ordem de Serviço.
- Converter e avaliar a lógica das consultas relacionais originais, explorando as vantagens da desnormalização de dados por meio de arrays embutidos e comandos nativos do MongoDB.

Esses objetivos norteiam a estruturação lógica do banco de dados e servem como base para o desenvolvimento de um sistema confiável, prático e alinhado com as necessidades de gestão de centros automotivos de

pequeno e médio porte.

1.2 Justificativa

Auto centers frequentemente enfrentam dificuldades em manter registros organizados de serviços prestados, peças utilizadas e informações de clientes. A utilização de um sistema informatizado pode reduzir falhas humanas, evitar perdas de dados e melhorar a produtividade. O projeto visa resolver esses problemas por meio de um banco de dados NoSQL .

O desenvolvimento deste projeto justifica-se pela oportunidade de aplicar, em um cenário de negócio real e factível, as lógicas de extração e consolidação de dados em ambos os paradigmas. Esse processo consolida o aprendizado prático no curso de Análise e Desenvolvimento de Sistemas, promovendo o raciocínio analítico necessário para adequar soluções tecnológicas às regras de negócio dinâmicas exigidas pelo mercado atual.

1.3 Aspectos Metodológicos

A presente pesquisa classifica-se como aplicada e exploratória, com abordagem quali-quantitativa baseada em um estudo de caso simulado (gestão de Auto Center). A execução técnica e prática ocorreu nas seguintes fases:

- Modelagem e Implementação Relacional (Fase Anterior): Estruturação conceitual e lógica das entidades sob a ótica da normalização (3FN). Criação do esquema físico no SQL Server, inserção de massa de dados e validação de relatórios gerenciais utilizando funções nativas (T-SQL).
- Modelagem Orientada a Documentos (Transição): Reavaliação lógica do modelo relacional para o paradigma NoSQL, aplicando o conceito

de desnormalização estratégica para definir quais dados seriam referenciados e quais seriam embutidos para otimizar o desempenho.

- Migração e Execução NoSQL (Fase Atual): Conversão definitiva das estruturas tabulares em coleções e documentos aninhados (BSON) no MongoDB, com a posterior refatoração das consultas analíticas para a sintaxe baseada em JavaScript, executada nativamente no shell do banco de dados.

1.4 Aporte Teórico

O desenvolvimento estrutural e conceitual deste trabalho fundamenta-se nos princípios clássicos da teoria de Banco de Dados. Utiliza-se como alicerce o rigor da Álgebra Relacional e as regras de normalização para garantir a integridade referencial no modelo SQL inicial, apoiando-se na literatura consolidada da área, a exemplo dos conceitos propostos por acadêmicos como Peter Rob e Carlos Coronel. Para a fundamentação do paradigma orientado a documentos, a pesquisa apoia-se nos princípios arquiteturais do NoSQL, especificamente na teoria de desnormalização de dados e no desempenho de leitura otimizado por estruturas chave-valor e vetores de documentos embutidos (JSON/BSON), garantindo que as métricas e relatórios gerados possuam total validade técnica e operacional.

Os bancos de dados NoSQL representam uma forma de armazenamento diferente dos bancos de dados relacionais tradicionais, pois utilizam estruturas de dados mais flexíveis e dinâmicas. Segundo Sadalage e Fowler (2013), o termo NoSQL não significa a ausência de uma linguagem de consulta, mas sim que esses sistemas não dependem exclusivamente do SQL e do modelo de tabelas para organizar e acessar os dados.

O crescimento dos bancos de dados NoSQL ocorreu no início dos anos 2000, quando grandes empresas de tecnologia, como Google, Amazon e Facebook, passaram a enfrentar dificuldades para armazenar e processar grandes quantidades de informações utilizando bancos de dados relacionais. Essas empresas precisavam de sistemas capazes de lidar com enormes volumes de dados, muitas vezes na escala de petabytes, e com maior

capacidade de escalabilidade.

Nesse contexto, o MongoDB destaca-se por adotar o modelo orientado a documentos. Os dados são armazenados em BSON (Binary JSON), um formato binário derivado do JSON que permite a inclusão de listas (arrays) e objetos aninhados em um mesmo documento. Essa característica reduz drasticamente ou até elimina a necessidade de realizar cruzamentos complexos de dados fragmentados (os tradicionais JOINS do modelo relacional), uma vez que as informações acessadas em conjunto já ficam armazenadas no mesmo documento.

Essa característica diferencia o MongoDB dos bancos de dados relacionais, nos quais as informações geralmente são divididas em várias tabelas para evitar a redundância de dados, seguindo as regras de normalização.

3 METODOLOGIA

3.1 Considerações Iniciais

O desenvolvimento deste projeto prático consolida a aplicação de conceitos avançados de persistência de dados, focando na reestruturação arquitetural de um sistema de gestão para um Auto Center. Inicialmente concebido e implementado sob o paradigma relacional, o banco de dados original fragmentava as informações de clientes, veículos, serviços e peças em tabelas normalizadas. A presente etapa metodológica consiste na migração e adaptação deste cenário para um ambiente não relacional, visando explorar na prática as vantagens de flexibilidade e o desempenho de leitura proporcionados por estruturas desnormalizadas, requisitos cada vez mais presentes no desenvolvimento moderno de sistemas.

3.2 O Modelo NoSQL

O termo "NoSQL" refere-se a bancos de dados que deixam de lado o modelo tradicional de tabelas para adotar estruturas mais flexíveis e dinâmicas. Apesar do nome, esses sistemas não abandonam por completo as consultas estruturadas, podendo ser acessados por meio de APIs ou

ferramentas como Query by Example (consulta por exemplo). Por isso, a sigla é mais conhecida no mercado tecnológico como Not Only SQL (Não Apenas SQL).

Esse foi o modelo escolhido para o desenvolvimento deste projeto. Diferente do banco de dados relacional, que exige estruturas rígidas e tabelas dependentes umas das outras, o NoSQL oferece várias opções de armazenamento (como chave-valor, grafos, colunas e documentos). A principal razão para utilizarmos esse modelo aqui é a sua capacidade de guardar informações relacionadas juntas, em um único lugar. Isso elimina a necessidade de fazer cruzamentos complexos entre várias tabelas só para juntar os dados de um atendimento ou serviço.

3.3 SGBD Utilizado e o Modelo de Dados

O SGBD escolhido foi o MongoDB que utiliza o modelo orientado a documentos. Nele, a estrutura tradicional do banco de dados relacional é deixada de lado em favor de conceitos muito mais flexíveis. Em vez de organizarmos as informações em "tabelas" e "linhas", o MongoDB agrupa os dados em "coleções" (collections) e "documentos" (documents).

Esses documentos são gravados no sistema em um formato chamado BSON (uma versão binária do JSON). A grande vantagem do BSON é que ele permite guardar tipos de dados mais complexos com facilidade, como datas, números decimais e, principalmente, listas e objetos.

Para realizar a interação com o banco de dados durante este projeto, além da execução de comandos via terminal (shell), foi utilizado o MongoDB Compass. Trata-se de uma interface gráfica poderosa para query, aggregation e análise de dados MongoDB em um ambiente visual. O uso dessa ferramenta oficial desempenhou um papel fundamental no desenvolvimento prático do trabalho, pois permitiu visualizar a estrutura dos documentos BSON de forma interativa e facilitou a montagem e o teste das consultas mais complexas.

A característica mais poderosa desse modelo, e que aplicamos diretamente no projeto prático do Auto Center, é a capacidade de aninhar

dados (usar documentos embutidos). No mundo relacional, havia a obrigatoriedade de separar as informações da Ordem de Serviço e das Peças em várias tabelas diferentes, conectando tudo depois através de chaves estrangeiras.

Já com o modelo de dados do MongoDB e o auxílio visual do Compass, foi possível embutir a lista (array) das peças consumidas diretamente dentro do próprio documento da Ordem de Serviço. O resultado prático é que, quando o sistema precisa consultar um atendimento, o banco traz todas as informações de uma só vez, em um bloco único. Isso elimina a necessidade de realizar cruzamentos de dados constantes, deixando as consultas mais rápidas e o desenvolvimento do sistema mais eficiente.

4. RESULTADOS OBTIDOS

Esta seção apresenta os resultados práticos do projeto de banco de dados desenvolvido utilizando o SGBD NoSQL MongoDB. O ambiente construído reflete estruturalmente as regras de negócio e as necessidades operacionais do Auto Center concebido inicialmente na etapa do Projeto de Banco de Dados Relacional.

Através da modelagem orientada a documentos, a base de dados foi populada com coleções essenciais (como Clientes, Veículos e Ordens de Serviço), aplicando os conceitos de desnormalização e vetores embutidos (arrays) para otimizar o desempenho das operações. Para validar a eficácia deste modelo, foram desenvolvidas e executadas 30 consultas operacionais e analíticas no terminal interativo do MongoDB (mongosh).

As consultas abrangem desde operações básicas de CRUD (Create, Read, Update, Delete) até lógicas avançadas utilizando o Aggregation Framework e blocos de decisão estruturada em JavaScript.

4.1 Criação das Coleções

```
use AutoCenter_NoSQL;
```

```
db.createCollection("clientes")  
db.createCollection("agendamentos")
```

```
db.createCollection("funcionarios")
db.createCollection("ordens_servico")
db.createCollection("pecas")
db.createCollection("servicos")
```

4.2 Inserção de dados

Inserção de Clientes

```
db.clientes.insertMany([
  {
    _id: 1,
    nome: "Bruno Henrique da Silva Balbino",
    telefone: "12991112222",
    email: "bruno.balbino@aluno.ifsp.edu.br",
    endereco: "Av. Central, 100",
    veiculos: [
      { placa: "ABC1D23", marca: "Volkswagen", modelo: "Gol 1.0 MSI", ano:
2020 },
      { placa: "BRA2E26", marca: "Honda", modelo: "Civic EXL", ano: 2019 }
    ]
  },
  {
    _id: 2,
    nome: "Carlos Henrique Cândido",
    telefone: "12988887777",
    email: "carlos.candido@email.com",
    endereco: "Rua das Flores, 45",
    veiculos: [
      { placa: "XYZ9G87", marca: "Chevrolet", modelo: "Onix Turbo", ano:
2022 }
    ]
  },
  {
    _id: 3,
    nome: "Ana Julia Costa",
    telefone: "11977776666",
    email: "anajulia@provedor.com",
    endereco: "Av. Paulista, 1500",
    veiculos: [
      { placa: "MNO5K44", marca: "Fiat", modelo: "Uno Mille", ano: 2013 }
    ]
  },
  {
    _id: 4,
    nome: "Mariana Souza Rodrigues",
    telefone: "12981234567",
    email: "mariana.souza@email.com",
    endereco: "Rua Principal, 200",
```

```

    veiculos: [
      { placa: "KJD4E12", marca: "Toyota", modelo: "Corolla XEI", ano: 2021 }
    ]
  },
  {
    _id: 5,
    nome: "Lucas lemes",
    telefone: "129887887777",
    email: "lucas.123@gmail.com",
    endereco: "Nova luis, nº 56",
    veiculos: [
      { placa: "LPO9J88", marca: "Ford", modelo: "Ka SE 1.0", ano: 2018 }
    ]
  },
  {
    _id: 6,
    nome: "Roberto Almeida Prado",
    telefone: "12992345678",
    email: "roberto.prado@gmail.com",
    endereco: "Alameda dos Pinheiros, 88",
    veiculos: []
  },
  {
    _id: 7,
    nome: "Lucas Luiz",
    telefone: "12988887777",
    email: "lucas.12@gmail.com",
    endereco: "Nova, nº 456",
    veiculos: []
  }
]);

```

Inserção dos Funcionários

```

db.funcionarios.insertMany([
  { _id: 1, nome: "Marcos Roberto Silva", cpf: "111.222.333-44", cargo:
"Mecânico Master" },
  { _id: 2, nome: "João Souza Oliveira", cpf: "222.333.444-55", cargo:
"Alinhador Técnico" },
  { _id: 3, nome: "Ricardo Dias Ferreira", cpf: "333.444.555-66", cargo:
"Eletricista Automotivo" },
  { _id: 4, nome: "Beatriz Ramos Garcia", cpf: "444.555.666-77", cargo:
"Mecânica Preventiva" }
]);

```

Inserção Serviços

```

db.servicos.insertMany([
  { _id: 1, descricao: "Troca de Óleo e Filtros", valor_mao_de_obra: 150.00 },
  { _id: 2, descricao: "Alinhamento 3D e Balanceamento",
    valor_mao_de_obra: 120.00 },
  { _id: 3, descricao: "Diagnóstico de Injeção Eletrônica", valor_mao_de_obra:
    200.00 },
  { _id: 4, descricao: "Troca de Pastilhas de Freio", valor_mao_de_obra:
    90.00 },
  { _id: 5, descricao: "Limpeza de Bicos Injetores", valor_mao_de_obra:
    180.00 }
]);

```

Inserção de Peças

```

db.pecas.insertMany([
  { _id: 1, nome: "Óleo Sintético 5W30 1L", descricao: "Lubrificante de alta
    performance", valor_unitario: 55.00, estoque_atual: 120 },
  { _id: 2, nome: "Filtro de Óleo Lubrificante", descricao: "Filtro blindado",
    valor_unitario: 35.00, estoque_atual: 45 },
  { _id: 3, nome: "Pastilha de Freio Dianteira", descricao: "Pastilha cerâmica",
    valor_unitario: 180.00, estoque_atual: 15 },
  { _id: 4, nome: "Jogo de Velas de Ignição", descricao: "Kit com 4 velas
    Iridium", valor_unitario: 220.00, estoque_atual: 20 },
  { _id: 5, nome: "Filtro de Combustível Flex", descricao: "Filtro de linha",
    valor_unitario: 40.00, estoque_atual: 30 },
  { _id: 6, nome: "Aditivo de Arrefecimento 1L", descricao: "Aditivo
    concentrado", valor_unitario: 28.00, estoque_atual: 60 }
]);

```

Inserção de Agendamentos

```

db.agendamentos.insertMany([
  { _id: 1, data_agendamento: new Date("2026-05-25T09:00:00Z"), horario:
    "09:00", cliente_id: 1, placa_veiculo: "ABC1D23" },
  { _id: 2, data_agendamento: new Date("2026-05-25T14:00:00Z"), horario:
    "14:00", cliente_id: 3, placa_veiculo: "MNO5K44" },
  { _id: 3, data_agendamento: new Date("2026-05-26T10:30:00Z"), horario:
    "10:30", cliente_id: 4, placa_veiculo: "KJD4E12" },
  { _id: 4, data_agendamento: new Date("2026-05-27T08:00:00Z"), horario:
    "08:00", cliente_id: 5, placa_veiculo: "LPO9J88" }
]);

```

Inserção de Ordens de Serviço

```

db.ordens_servico.insertMany([

```

```
{
  _id: 1,
  data_servico: new Date("2026-04-10T00:00:00Z"),
  status: "Concluída",
  cliente_id: 1,
  placa_veiculo: "ABC1D23",
  funcionario_nome: "Marcos Roberto Silva",
  itens_servico: [
    { servico_id: 1, descricao: "Troca de Óleo e Filtros", quantidade: 1,
valor_cobrado: 150.00 },
    { servico_id: 4, descricao: "Troca de Pastilhas de Freio", quantidade: 1,
valor_cobrado: 90.00 }
  ],
  itens_peca: [
    { peca_id: 1, nome: "Óleo Sintético 5W30 1L", quantidade: 4,
valor_venda: 55.00 },
    { peca_id: 2, nome: "Filtro de Óleo Lubrificante", quantidade: 1,
valor_venda: 35.00 },
    { peca_id: 3, nome: "Pastilha de Freio Dianteira", quantidade: 1,
valor_venda: 180.00 }
  ]
},
{
  _id: 2,
  data_servico: new Date("2026-04-15T00:00:00Z"),
  status: "Concluída",
  cliente_id: 2,
  placa_veiculo: "XYZ9G87",
  funcionario_nome: "João Souza Oliveira",
  itens_servico: [
    { servico_id: 2, descricao: "Alinhamento 3D e Balanceamento",
quantidade: 1, valor_cobrado: 120.00 }
  ],
  itens_peca: []
},
{
  _id: 3,
  data_servico: new Date("2026-05-02T00:00:00Z"),
  status: "Em Andamento",
  cliente_id: 3,
  placa_veiculo: "MNO5K44",
  funcionario_nome: "Ricardo Dias Ferreira",
  itens_servico: [
    { servico_id: 3, descricao: "Diagnóstico de Injeção Eletrônica",
quantidade: 1, valor_cobrado: 200.00 }
  ],
  itens_peca: [
    { peca_id: 4, nome: "Jogo de Velas de Ignição", quantidade: 1,
valor_venda: 220.00 }
  ]
},
}
```

```

{
  _id: 4,
  data_servico: new Date("2026-05-18T00:00:00Z"),
  status: "Pendente",
  cliente_id: 4,
  placa_veiculo: "KJD4E12",
  funcionario_nome: "Marcos Roberto Silva",
  itens_servico: [],
  itens_peca: []
},
{
  _id: 5,
  data_servico: new Date("2026-05-22T00:00:00Z"),
  status: "Concluída",
  cliente_id: 1,
  placa_veiculo: "BRA2E26",
  funcionario_nome: "Beatriz Ramos Garcia",
  itens_servico: [
    { servico_id: 1, descricao: "Troca de Óleo e Filtros", quantidade: 1,
    valor_cobrado: 140.00 },
    { servico_id: 5, descricao: "Limpeza de Bicos Injetores", quantidade: 1,
    valor_cobrado: 180.00 }
  ],
  itens_peca: [
    { peca_id: 1, nome: "Óleo Sintético 5W30 1L", quantidade: 4,
    valor_venda: 55.00 },
    { peca_id: 2, nome: "Filtro de Óleo Lubrificante", quantidade: 1,
    valor_venda: 35.00 }
  ]
},
{
  _id: 6,
  data_servico: new Date("2026-05-23T00:00:00Z"),
  status: "Pendente",
  cliente_id: 5,
  placa_veiculo: "LPO9J88",
  funcionario_nome: "João Souza Oliveira",
  itens_servico: [],
  itens_peca: []
}
]);

```

3.4 Matriz de 30 Consultas NoSQL

1. Clientes com Domínio de E-mail Acadêmico (IFSP).
 Filtra os clientes que possuem e-mail institucional utilizando Expressões Regulares (Regex) para avaliar e buscar o padrão de texto diretamente nos

documentos da coleção.

```
db.clientes.find(  
  { email: /ifsp\.edu\.br/i },  
  { _id: 0, nome: 1, email: 1 }  
)
```

```
{  
  nome: 'Bruno Henrique da Silva Balbino',  
  email: 'bruno.balbino@aluno.ifsp.edu.br'  
}
```

2. Veículos mais novos ordenados por Ano.

Lista os carros do mais novo para o mais antigo.

```
db.clientes.aggregate([  
  { $unwind: "$veiculos" },  
  { $project: {  
    _id: 0,  
    placa: "$veiculos.placa",  
    marca: "$veiculos.marca",  
    modelo: "$veiculos.modelo",  
    ano: "$veiculos.ano"  
  }},  
  { $sort: { ano: -1 } }
```

1)

```
{
  placa: 'XYZ9G87',
  marca: 'Chevrolet',
  modelo: 'Onix Turbo',
  ano: 2022
}
{
  placa: 'KJD4E12',
  marca: 'Toyota',
  modelo: 'Corolla XEI',
  ano: 2021
}
{
  placa: 'ABC1D23',
  marca: 'Volkswagen',
  modelo: 'Gol 1.0 MSI',
  ano: 2020
}
{
  placa: 'BRA2E26',
  marca: 'Honda',
  modelo: 'Civic EXL',
  ano: 2019
}
{
  placa: 'LP09J88',
  marca: 'Ford',
  modelo: 'Ka SE 1.0',
  ano: 2018
}
{
  placa: 'MN05K44',
  marca: 'Fiat',
  modelo: 'Uno Mille',
  ano: 2013
},
```

3. Filtro Combinado de Peças com Estoque Crítico ou Alto Valor.

Filtra os documentos da coleção de peças utilizando o operador lógico \$or combinado com os operadores de comparação \$lt (menor que) e \$gt (maior que), retornando apenas os campos essenciais através da projeção.

```
db.pecas.find(  
  {  
    $or: [  
      { estoque_atual: { $lt: 30 } },  
      { valor_unitario: { $gt: 150.00 } }  
    ]  
  },  
  { _id: 1, nome: 1, valor_unitario: 1, estoque_atual: 1 }  
)
```

```
{  
  _id: 3,  
  nome: 'Pastilha de Freio Dianteira',  
  valor_unitario: 180,  
  estoque_atual: 15  
}  
{  
  _id: 4,  
  nome: 'Jogo de Velas de Ignição',  
  valor_unitario: 220,  
  estoque_atual: 20  
}
```

4. Seleção Limitada dos Serviços Mais Caros da Oficina.

Retorna apenas os 3 serviços com maior valor de mão de obra cadastrados na base de dados, utilizando o método .sort() para ordenar de forma decrescente e o método .limit() para restringir a quantidade de documentos exibidos.

```
db.servicos.find(  
  {},  
  { _id: 1, descricao: 1, valor_mao_de_obra: 1 }
```

```
).sort({ valor_mao_de_obra: -1 }).limit(3)
```

```
{
  _id: 3,
  descricao: 'Diagnóstico de Injeção Eletrônica',
  valor_mao_de_obra: 200
}
{
  _id: 5,
  descricao: 'Limpeza de Bicos Injetores',
  valor_mao_de_obra: 180
}
{
  _id: 1,
  descricao: 'Troca de Óleo e Filtros',
  valor_mao_de_obra: 150
}
```

5. Busca de Peças em Faixas Específicas de Preço.

Filtra os documentos da coleção de peças cujo valor unitário esteja em um intervalo fechado (entre R\$ 30,00 e R\$ 100,00).

```
db.pecas.find(
  { valor_unitario: { $gte: 30.00, $lte: 100.00 } },
  { _id: 0, nome: 1, valor_unitario: 1 }
)
```

```

{
  nome: 'Óleo Sintético 5W30 1L',
  valor_unitario: 55
}
{
  nome: 'Filtro de Óleo Lubrificante',
  valor_unitario: 35
}
{
  nome: 'Filtro de Combustível Flex',
  valor_unitario: 40
}

```

6. Relatório de Veículos e seus Respectivos Proprietários.
Combina os dados estruturados no banco para mapear quem é o dono de cada carro cadastrado.

```

db.clientes.aggregate([
  { $unwind: "$veiculos" },
  { $project: {
    _id: 0,
    Placa: "$veiculos.placa",
    Marca: "$veiculos.marca",
    Modelo: "$veiculos.modelo",
    Nome_Proprietario: "$nome"
  }}
])

```

```
{
  Placa: 'ABC1D23',
  Marca: 'Volkswagen',
  Modelo: 'Gol 1.0 MSI',
  Nome_Proprietario: 'Bruno Henrique da Silva Balbino'
}
{
  Placa: 'BRA2E26',
  Marca: 'Honda',
  Modelo: 'Civic EXL',
  Nome_Proprietario: 'Bruno Henrique da Silva Balbino'
}
{
  Placa: 'XYZ9G87',
  Marca: 'Chevrolet',
  Modelo: 'Onix Turbo',
  Nome_Proprietario: 'Carlos Henrique Cândido'
}
{
  Placa: 'MN05K44',
  Marca: 'Fiat',
  Modelo: 'Uno Mille',
  Nome_Proprietario: 'Ana Julia Costa'
}
{
  Placa: 'KJD4E12',
  Marca: 'Toyota',
  Modelo: 'Corolla XEI',
  Nome_Proprietario: 'Mariana Souza Rodrigues'
}
{
  Placa: 'LP09J88',
  Marca: 'Ford',
  Modelo: 'Ka SE 1.0',
  Nome_Proprietario: 'Lucas Lemes'
}
```

7. Histórico Completo de Agendamentos da Oficina com Detalhes.

Une os dados de agendamentos com a coleção de clientes para que a recepção controle a fila de entrada exibindo o nome do proprietário e o modelo do veículo.

```
db.agendamentos.aggregate([
  {
    $lookup: {
      from: "clientes",
      localField: "cliente_id",
      foreignField: "_id",
      as: "cliente_info"
    }
  },
  { $unwind: "$cliente_info" },
  { $unwind: "$cliente_info.veiculos" },
  {
    $match: {
      $expr: { $eq: ["$placa_veiculo", "$cliente_info.veiculos.placa"] }
    }
  },
  {
    $project: {
      _id: 0,
      ID_AGENDAMENTO: "$_id",
      DATA_AGENDAMENTO: "$data_agendamento",
      HORARIO: "$horario",
      NOME: "$cliente_info.nome",
      MODELO: "$cliente_info.veiculos.modelo"
    }
  }
])
```

```

{
  ID_AGENDAMENTO: 1,
  DATA_AGENDAMENTO: 2026-05-25T09:00:00.000Z,
  HORARIO: '09:00',
  NOME: 'Bruno Henrique da Silva Balbino',
  MODELO: 'Gol 1.0 MSI'
}
{
  ID_AGENDAMENTO: 2,
  DATA_AGENDAMENTO: 2026-05-25T14:00:00.000Z,
  HORARIO: '14:00',
  NOME: 'Ana Julia Costa',
  MODELO: 'Uno Mille'
}
{
  ID_AGENDAMENTO: 3,
  DATA_AGENDAMENTO: 2026-05-26T10:30:00.000Z,
  HORARIO: '10:30',
  NOME: 'Mariana Souza Rodrigues',
  MODELO: 'Corolla XEI'
}
{
  ID_AGENDAMENTO: 4,
  DATA_AGENDAMENTO: 2026-05-27T08:00:00.000Z,
  HORARIO: '08:00',
  NOME: 'Lucas Lemes',
  MODELO: 'Ka SE 1.0'
}

```

8. Mapeamento de Clientes Sem Ordens de Serviço.
 Identifica clientes que se cadastraram ou agendaram, mas ainda não fecharam ordens de serviço físicas no auto center.

```

db.clientes.aggregate([
  {
    $lookup: {
      from: "ordens_servico",
      localField: "_id",
      foreignField: "cliente_id",

```

```

        as: "historico_os"
      }
    },
    {
      $match: {
        historico_os: { $size: 0 }
      }
    },
    {
      $project: {
        _id: 0,
        ID_CLIENTE: "$_id",
        NOME: "$nome"
      }
    }
  ]
})

```

```

{
  ID_CLIENTE: 6,
  NOME: 'Roberto Almeida Prado'
}
{
  ID_CLIENTE: 7,
  NOME: 'Lucas Luiz'
}

```

9.Quantidade de Veículos Cadastrados por Cliente (Ordenado).
Agrega os registros, conta o tamanho da frota de cada cliente e ordena o resultado final pelo nome.

```

db.clientes.aggregate([
  { $unwind: "$veiculos" },
  { $group: {
    _id: "$nome",
    Total_Veiculos: { $sum: 1 }
  } },
  { $project: {
    _id: 0,
    NOME: "$_id",
    Total_Veiculos: 1
  } },
  { $sort: { NOME: 1 } }
])

```

```

{
  Total_Veiculos: 1,
  NOME: 'Ana Julia Costa'
}
{
  Total_Veiculos: 2,
  NOME: 'Bruno Henrique da Silva Balbino'
}
{
  Total_Veiculos: 1,
  NOME: 'Carlos Henrique Cândido'
}
{
  Total_Veiculos: 1,
  NOME: 'Lucas lemes'
}
{
  Total_Veiculos: 1,
  NOME: 'Mariana Souza Rodrigues'
}

```

10. Valor Médio das Mãos de Obra dos Serviços Cadastrados.
 Calcula a média geral de custos da mão de obra cobrada pela oficina.

```

db.servicos.aggregate([
  {
    $group: {
      _id: null,
      mediaBruta: { $avg: "$valor_mao_de_obra" }
    }
  },
  {
    $project: {
      _id: 0,
      "Média da Mão de Obra R$": { $round: ["$mediaBruta", 2] }
    }
  }
])

```

```
{
  'Média da Mão de Obra R$': 148
}
```

11. Ranking de Produtividade dos Funcionários.

Exibe os mecânicos que possuem mais de 1 ordem de serviço vinculada sob sua responsabilidade, ordenados do mais produtivo para o menos, usando um estágio \$match executado após o agrupamento (\$group). O agrupamento é feito na própria coleção de ordens de serviço, contabilizando a frequência de cada funcionario_nome.

```
db.ordens_servico.aggregate([
  {
    $group: {
      _id: "$funcionario_nome",
      Qtd_Ordens_Executadas: { $sum: 1 }
    }
  },
  {
    $match: {
      Qtd_Ordens_Executadas: { $gt: 1 }
    }
  },
  {
    $project: {
      _id: 0,
      NOME: "$_id",
      Qtd_Ordens_Executadas: 1
    }
  },
  {
    $sort: { NOME: 1 }
  }
])
```

```
{
  Qtd_Ordens_Executadas: 2,
  NOME: 'João Souza Oliveira'
}
{
  Qtd_Ordens_Executadas: 2,
  NOME: 'Marcos Roberto Silva'
}
```

12.União de Clientes e Funcionários.

Cria uma lista única combinando nomes cadastrados no sistema em duas coleções distintas, removendo eventuais duplicidades.

```
db.clientes.aggregate([

  {
    $project: {
      _id: 0,
      "Nome do Cadastro": "$nome",
      Tipo: "Cliente"
    }
  },
  {
    $unionWith: {
      coll: "funcionarios",
      pipeline: [
        {
          $project: {
            _id: 0,
            "Nome do Cadastro": "$nome",
            Tipo: "Funcionario"
          }
        }
      ]
    }
  },
  {
    $group: {
      _id: { nome: "$Nome do Cadastro", tipo: "$Tipo" }
    }
  },
  {
    $project: {
      _id: 0,
      "Nome do Cadastro": "$_id.nome",
      Tipo: "$_id.tipo"
    }
  },
  {
    $sort: { "Nome do Cadastro": 1 }
  }
])
```

```
{
  'Nome do Cadastro': 'Ana Julia Costa',
  Tipo: 'Cliente'
}
{
  'Nome do Cadastro': 'Beatriz Ramos Garcia',
  Tipo: 'Funcionario'
}
{
  'Nome do Cadastro': 'Bruno Henrique da Silva Balbino',
  Tipo: 'Cliente'
}
{
  'Nome do Cadastro': 'Carlos Henrique Cândido',
  Tipo: 'Cliente'
}
{
  'Nome do Cadastro': 'João Souza Oliveira',
  Tipo: 'Funcionario'
}
{
  'Nome do Cadastro': 'Lucas Luiz',
  Tipo: 'Cliente'
}
{
  'Nome do Cadastro': 'Lucas lemes',
  Tipo: 'Cliente'
}
{
  'Nome do Cadastro': 'Marcos Roberto Silva',
  Tipo: 'Funcionario'
}
{
  'Nome do Cadastro': 'Mariana Souza Rodrigues',
  Tipo: 'Cliente'
}
```

13.Verificação de Coincidência de Nomes com Retorno Condicional Nativo.
Valida se existe algum funcionário que também está cadastrado como cliente na oficina.

```
db.clientes.aggregate([

  {
    $lookup: {
      from: "funcionarios",
      localField: "nome",
      foreignField: "nome",
      as: "nome_coincidente"
    }
  },
  {
    $group: {
      _id: null,
      coincidencias: {
        $push: {
          $cond: [ { $gt: [ { $size: "$nome_coincidente" }, 0 ] }, "$nome",
"$REMOVE" ]
        }
      }
    }
  },
  {
    $project: {
      _id: 0,
      Status: {
        $cond: {
          if: { $eq: [ { $size: "$coincidencias" }, 0 ] },
          then: "NÃO TEM FUNCIONÁRIOS CADASTRADOS COMO
CLIENTES.",
          else: "Coincidência encontrada nos cadastros!"
        }
      },
      Nomes_Encontrados: {
        $cond: {
          if: { $eq: [ { $size: "$coincidencias" }, 0 ] },
          then: "Nenhum",
          else: "$coincidencias"
        }
      }
    }
  }
])
```

```
{
  Status: 'NÃO TEM FUNCIONÁRIOS CADASTRADOS COMO CLIENTES.',
  Nomes_Encontrados: 'Nenhum'
}
```

14. Clientes que Nunca Tiveram um Veículo Cadastrado.
Retorna os clientes registrados na base que não possuem automóveis vinculados ao seu cadastro.

```
db.clientes.aggregate([
  {
    $match: {
      veiculos: { $size: 0 }
    }
  },
  {
    $project: {
      _id: 0,
      "Código do Cliente": "$_id",
      "NOME": "$nome"
    }
  }
])
```

```
{
  'Código do Cliente': 6,
  NOME: 'Roberto Almeida Prado'
}
{
  'Código do Cliente': 7,
  NOME: 'Lucas Luiz'
}
```

15. Peças Aplicadas na OS Nº 1
Busca os nomes das peças e valores aplicados na Ordem de Serviço 1.

```
db.ordens_servico.find(
```

```

    { _id: 1 },
    {
      _id: 0,
      "itens_peca.nome": 1,
      "itens_peca.valor_venda": 1
    }
  )

```

```

{
  itens_peca: [
    {
      nome: 'Óleo Sintético 5W30 1L',
      valor_venda: 55
    },
    {
      nome: 'Filtro de Óleo Lubrificante',
      valor_venda: 35
    },
    {
      nome: 'Pastilha de Freio Dianteira',
      valor_venda: 180
    }
  ]
}

```

16. Filtro de Ordens de Serviço com Valor Superior à Média de Serviços. Encontra itens faturados em ordens de serviço cujo valor da mão de obra superou a média geral da oficina. O valor médio é calculado através de um estágio \$group e armazenado em uma variável, depois o operador \$unwind para separar os itens aninhados e filtra os resultados utilizando o operador \$gt comparando com a variável gerada.

```

var resultadoMedia = db.servicos.aggregate([
  { $group: { _id: null, media: { $avg: "$valor_mao_de_obra" } } }
]).toArray();

```

```

var mediaGeral = resultadoMedia[0].media;
db.ordens_servico.aggregate([
  { $unwind: "$itens_servico" },

```

```

{
  $match: {
    "itens_servico.valor_cobrado": { $gt: mediaGeral }
  }
},
{
  $project: {
    _id: 0,
    ID_OS: "$_id",
    VALOR_COBRADO: "$itens_servico.valor_cobrado"
  }
}
])

```

```

{
  ID_OS: 1,
  VALOR_COBRADO: 150
}
{
  ID_OS: 3,
  VALOR_COBRADO: 200
}
{
  ID_OS: 5,
  VALOR_COBRADO: 180
}

```

17. Verificar Agendamentos.

Seleciona os clientes que possuem agendamentos confirmados batendo as chaves com uma subconsulta.. A subconsulta é resolvida na camada de execução (mongosh) utilizando o método `.distinct()` na coleção de agendamentos para extrair uma lista única de IDs. Esse array é então repassado para o filtro principal na coleção de clientes.

```

var clientesComAgendamento = db.agendamentos.distinct("cliente_id");

db.clientes.find(
  { _id: { $in: clientesComAgendamento } },
  { _id: 0, nome: 1 }
)

```

```
{
  nome: 'Bruno Henrique da Silva Balbino'
}
{
  nome: 'Ana Julia Costa'
}
{
  nome: 'Mariana Souza Rodrigues'
}
{
  nome: 'Lucas Lemes'
}
```

18. Comparação de Preço Máximo .

Encontra as peças cujo valor unitário supera todos os valores cobrados individualmente no array embutido de serviços para um serviço específico.

```
var resultadoAll = db.ordens_servico.aggregate([
  { $unwind: "$itens_servico" },
  {
    $match: { "itens_servico.servico_id": 5 }
  },
  {
    $group: {
      _id: null,
      valor_maximo: { $max: "$itens_servico.valor_cobrado" }
    }
  }
]).toArray();
```

```
var valorTeto = resultadoAll.length > 0 ? resultadoAll[0].valor_maximo : 0;
```

```
db.pecas.find(
  { valor_unitario: { $gt: valorTeto } },
  { _id: 0, nome: 1, valor_unitario: 1 }
)
```

```
{
  nome: 'Jogo de Velas de Ignição',
  valor_unitario: 220
}
```

19. Criação da View de Faturamento Generalizado das OSs.

Guarda a lógica complexa de consolidação financeira em uma estrutura virtual estável. No MongoDB, as visões de leitura são criadas com o comando `db.createView()`, que salva um pipeline de agregação e o executa sempre que os dados são consultados.

```
db.createView(
  "Vw_Faturamento_Ordens_Servico",
  "ordens_servico",
  [
    {
      $lookup: {
        from: "clientes",
        localField: "cliente_id",
        foreignField: "_id",
        as: "cliente_info"
      }
    },
    { $unwind: "$cliente_info" },
    {
      $project: {
        _id: 0,
        ID_OS: "$_id",
        DATA_SERVICO: "$data_servico",
        STATUS_OS: "$status",
        Nome_Cliente: "$cliente_info.nome",
        Valor_Servicos: {
          $sum: {
            $map: {
              input: { $ifNull: ["$itens_servico", []] },
              as: "item",
              in: {
                $multiply: [
                  { $ifNull: ["$$item.quantidade", 1] },
                  { $ifNull: ["$$item.valor_cobrado", 0] }
                ]
              }
            }
          }
        }
      }
    },
    {
      Valor_Pecas: {
        $sum: {
          $map: {
            input: { $ifNull: ["$itens_peca", []] },
            as: "item",
            in: {
              $multiply: [
                { $ifNull: ["$$item.quantidade", 1] },
                { $ifNull: ["$$item.valor_venda", 0] }
              ]
            }
          }
        }
      }
    }
  ]
)
```

```

    }
  }
},
{
  $addField: {
    Valor_Total_OS: { $add: ["$Valor_Servicos", "$Valor_Pecas"] }
  }
}
]
)

```

```
{ ok: 1 }
```

20. Consulta Direta Baseada na View de Faturamento

Retorna apenas faturamentos de ordens de serviço concluídas consultando a View criada.

```

db.Vw_Faturamento_Ordens_Servico.find(
  { STATUS_OS: "Concluída" },
  { _id: 0, ID_OS: 1, Nome_Cliente: 1, Valor_Total_OS: 1 }
)

```

```

{
  ID_OS: 1,
  Nome_Cliente: 'Bruno Henrique da Silva Balbino',
  Valor_Total_OS: 435
}
{
  ID_OS: 2,
  Nome_Cliente: 'Carlos Henrique Cândido',
  Valor_Total_OS: 0
}
{
  ID_OS: 5,
  Nome_Cliente: 'Bruno Henrique da Silva Balbino',
  Valor_Total_OS: 255
}

```

21. Tabela Temporária Local para Triagem de Agendamentos do Dia.
Cria uma estrutura temporária em memória para manipular dados de forma ágil na sessão

```

var FilaTrabalho = db.agendamentos.aggregate([
  {
    $lookup: {
      from: "clientes",
      localField: "cliente_id",
      foreignField: "_id",
      as: "dados_dono"
    }
  },
  {
    $unwind: { path: "$dados_dono", preserveNullAndEmptyArrays: true }
  },
  {
    $project: {
      _id: 0,
      Placa: "$placa_veiculo",
      Dono: "$dados_dono.nome"
    }
  }
])

```

```

    }
  }
  ]).toArray();

```

```

print("--- Fila de Trabalho Gerada ---");
printjson(FilaTrabalho);

```

```

FilaTrabalho = null;

```

```

--- Fila de Trabalho Gerada ---
[
  { Placa: 'ABC1D23', Dono: 'Bruno Henrique da Silva Balbino' },
  { Placa: 'MN05K44', Dono: 'Ana Julia Costa' },
  { Placa: 'KJD4E12', Dono: 'Mariana Souza Rodrigues' },
  { Placa: 'LP09J88', Dono: 'Lucas lemes' }
]

```

22. Formatação de Nomes e Extração de Iniciais.

Demonstra o uso de operadores nativos de manipulação de strings para a exibição de relatórios personalizados.

```

db.clientes.aggregate([
  {
    $project: {
      _id: 0,
      Nome_Maiusculo: { $toUpper: "$nome" },
      Abrevia_Nome: { $substrCP: ["$nome", 0, 3] },
      Info_Contato: { $concat: ["Contato: ", "$telefone"] }
    }
  }
])

```

```
{
  Nome_Maiusculo: 'BRUNO HENRIQUE DA SILVA BALBINO',
  Abrevia_Nome: 'Bru',
  Info_Contato: 'Contato: 12991112222'
}
{
  Nome_Maiusculo: 'CARLOS HENRIQUE CÂNDIDO',
  Abrevia_Nome: 'Car',
  Info_Contato: 'Contato: 12988887777'
}
{
  Nome_Maiusculo: 'ANA JULIA COSTA',
  Abrevia_Nome: 'Ana',
  Info_Contato: 'Contato: 11977776666'
}
{
  Nome_Maiusculo: 'MARIANA SOUZA RODRIGUES',
  Abrevia_Nome: 'Mar',
  Info_Contato: 'Contato: 12981234567'
}
{
  Nome_Maiusculo: 'LUCAS LEMES',
  Abrevia_Nome: 'Luc',
  Info_Contato: 'Contato: 129887887777'
}
{
  Nome_Maiusculo: 'ROBERTO ALMEIDA PRADO',
  Abrevia_Nome: 'Rob',
  Info_Contato: 'Contato: 12992345678'
}
{
  Nome_Maiusculo: 'LUCAS LUIZ',
  Abrevia_Nome: 'Luc',
  Info_Contato: 'Contato: 12988887777'
}
```

23.Arredondamento e Cálculos Financeiros de Margem.
Aplica simulações de reajuste utilizando funções matemáticas para arredondamento.

```
db.pecas.aggregate([
  {
    $project: {
      _id: 0,
      NOME: "$nome",
      Preco_Base: "$valor_unitario",
      Preco_Com_Aumento: {
        $round: [ { $multiply: ["$valor_unitario", 1.075] }, 2 ]
      },
      Preco_Teto_Inteiro: { $ceil: "$valor_unitario" }
    }
  }
])
```

```
{
  NOME: 'Óleo Sintético 5W30 1L',
  Preco_Base: 55,
  Preco_Com_Aumento: 59.12,
  Preco_Teto_Inteiro: 55
}
{
  NOME: 'Filtro de Óleo Lubrificante',
  Preco_Base: 35,
  Preco_Com_Aumento: 37.62,
  Preco_Teto_Inteiro: 35
}
{
  NOME: 'Pastilha de Freio Dianteira',
  Preco_Base: 180,
  Preco_Com_Aumento: 193.5,
  Preco_Teto_Inteiro: 180
}
{
  NOME: 'Jogo de Velas de Ignição',
  Preco_Base: 220,
  Preco_Com_Aumento: 236.5,
  Preco_Teto_Inteiro: 220
}
{
  NOME: 'Filtro de Combustível Flex',
  Preco_Base: 40,
  Preco_Com_Aumento: 43,
  Preco_Teto_Inteiro: 40
}
{
  NOME: 'Aditivo de Arrefecimento 1L',
  Preco_Base: 28,
  Preco_Com_Aumento: 30.1,
  Preco_Teto_Inteiro: 28
}
```

24.Extração de Componentes da Data da OS.

Usa operadores nativos para segmentar o momento em que as manutenções aconteceram. No MongoDB, a extração de componentes de um campo ISODate é feita no estágio de projeção (\$project).

```
db.ordens_servico.aggregate([  
  {  
    $project: {  
      _id: 0,  
      ID_OS: "$_id",  
      DATA_SERVICO: "$data_servico",  
      Dia_Abertura: { $dayOfMonth: "$data_servico" },  
      Mes_Abertura: { $month: "$data_servico" },  
      Ano_Abertura: { $year: "$data_servico" }  
    }  
  }  
])
```

```
{
  ID_OS: 1,
  DATA_SERVICO: 2026-04-10T00:00:00.000Z,
  Dia_Abertura: 10,
  Mes_Abertura: 4,
  Ano_Abertura: 2026
}
{
  ID_OS: 2,
  DATA_SERVICO: 2026-04-15T00:00:00.000Z,
  Dia_Abertura: 15,
  Mes_Abertura: 4,
  Ano_Abertura: 2026
}
{
  ID_OS: 3,
  DATA_SERVICO: 2026-05-02T00:00:00.000Z,
  Dia_Abertura: 2,
  Mes_Abertura: 5,
  Ano_Abertura: 2026
}
{
  ID_OS: 4,
  DATA_SERVICO: 2026-05-18T00:00:00.000Z,
  Dia_Abertura: 18,
  Mes_Abertura: 5,
  Ano_Abertura: 2026
}
{
  ID_OS: 5,
  DATA_SERVICO: 2026-05-22T00:00:00.000Z,
  Dia_Abertura: 22,
  Mes_Abertura: 5,
  Ano_Abertura: 2026
}
```

25. Tempo Decorrido Desde a Abertura da Ordem de Serviço.
Mede o intervalo de tempo em dias entre a data do serviço e a execução atual do banco de dados.

```
db.ordens_servico.aggregate([
{
  $project: {
    _id: 0,
    ID_OS: "$_id",
    DATA_SERVICO: "$data_servico",
    Dias_Desde_Abertura: {
      $dateDiff: {
        startDate: "$data_servico",
        endDate: "$$NOW",
        unit: "day"
      }
    }
  }
}
])
```

```
{
  ID_OS: 1,
  DATA_SERVICO: 2026-04-10T00:00:00.000Z,
  Dias_Desde_Abertura: Long('79')
}
{
  ID_OS: 2,
  DATA_SERVICO: 2026-04-15T00:00:00.000Z,
  Dias_Desde_Abertura: Long('74')
}
{
  ID_OS: 3,
  DATA_SERVICO: 2026-05-02T00:00:00.000Z,
  Dias_Desde_Abertura: Long('57')
}
{
  ID_OS: 4,
  DATA_SERVICO: 2026-05-18T00:00:00.000Z,
  Dias_Desde_Abertura: Long('41')
}
{
  ID_OS: 5,
  DATA_SERVICO: 2026-05-22T00:00:00.000Z,
  Dias_Desde_Abertura: Long('37')
}
{
  ID_OS: 6,
  DATA_SERVICO: 2026-05-23T00:00:00.000Z,
  Dias_Desde_Abertura: Long('36')
}
```

26. Análise de Envelhecimento das Ordens de Serviço (Lógica Condicional e Formatação de Data).

Esta consulta calcula há quantos dias cada Ordem de Serviço está aberta em relação à data atual, categorizando a urgência para auxiliar no controle de pendências.

```
db.ordens_servico.aggregate([
{
  $addFields: {
    dias_calculados: {
      $dateDiff: {
        startDate: "$data_servico",
        endDate: "$$NOW",
        unit: "day"
      }
    }
  }
},
{
  $project: {
    _id: 0,
    "Nº da OS": "$_id",
    "Estado da OS": "$status",
    "Data de Abertura": {
      $dateToString: { format: "%d/%m/%Y", date: "$data_servico" }
    },
    "Dias em Aberto": "$dias_calculados",
    "Classificação de Urgência": {
      $switch: {
        branches: [
          { case: { $lte: ["$dias_calculados", 2] }, then: "Regular (Até 2 dias)" },
          { case: { $lte: ["$dias_calculados", 5] }, then: "Atenção (3 a 5 dias)" }
        ],
        default: "CRÍTICO (Mais de 5 dias)"
      }
    }
  }
}]
```

```
}  
}  
,  
{  
  $sort: { "Dias em Aberto": -1 }  
}  
])
```

```

{
  'Nº da OS': 1,
  'Estado da OS': 'Concluída',
  'Data de Abertura': '10/04/2026',
  'Dias em Aberto': Long('79'),
  'Classificação de Urgência': 'CRÍTICO (Mais de 5 dias)'
}
{
  'Nº da OS': 2,
  'Estado da OS': 'Concluída',
  'Data de Abertura': '15/04/2026',
  'Dias em Aberto': Long('74'),
  'Classificação de Urgência': 'CRÍTICO (Mais de 5 dias)'
}
{
  'Nº da OS': 3,
  'Estado da OS': 'Em Andamento',
  'Data de Abertura': '02/05/2026',
  'Dias em Aberto': Long('57'),
  'Classificação de Urgência': 'CRÍTICO (Mais de 5 dias)'
}
{
  'Nº da OS': 4,
  'Estado da OS': 'Pendente',
  'Data de Abertura': '18/05/2026',
  'Dias em Aberto': Long('41'),
  'Classificação de Urgência': 'CRÍTICO (Mais de 5 dias)'
}
{
  'Nº da OS': 5,
  'Estado da OS': 'Concluída',
  'Data de Abertura': '22/05/2026',
  'Dias em Aberto': Long('37'),
  'Classificação de Urgência': 'CRÍTICO (Mais de 5 dias)'
}
}

```

27. Validação de Saldo via Estrutura de Decisão (IF...ELSE).
Exemplo procedural simples que avalia e imprime alertas sobre o estoque no

terminal.

```
var peca = db.pecas.findOne({ _id: 1 });

if (peca != null) {

    var EstoqueAtual = peca.estoque_atual;

    if (EstoqueAtual < 50) {
        print("ALERTA: Estoque de segurança atingido para o item 1!");
    } else {
        print("Saldo de estoque em conformidade regulatória.");
    }

} else {
    print("Erro: Peça não cadastrada no sistema.");
}
```



```
Saldo de estoque em conformidade regulatória.
```

28. Rotina Armazenada (Função JS) para Consulta de Ordens por Cliente. Uma rotina que recebe o ID de um cliente via parâmetro e extrai suas movimentações.

```
function usp_ConsultarOSCliente(IdCliente) {
    var resultado = db.ordens_servico.aggregate([
        {
            $match: { cliente_id: IdCliente }
        },
        {
            $project: {
                _id: 0,
                ID_OS: "$_id",
                DATA_SERVICO: "$data_servico",
                STATUS_OS: "$status",
                PLACA_VEICULO: "$placa_veiculo"
            }
        }
    ]).toArray();

    print("--- Movimentações Encontradas ---");
    printjson(resultado);
}
```

exemplo de uso:
usp_ConsultarOSCliente(1);

29. Função de Usuário (UDF Inline) para Listar Peças de uma OS.

Retorna um array de documentos contendo os insumos associados à Ordem de Serviço passada como argumento.

```
function fn_ListarPecasDaOS(IdOS) {  
    var resultado = db.ordens_servico.aggregate([  
        { $match: { _id: IdOS } },  
  
        { $unwind: "$itens_peca" },  
  
        {  
            $lookup: {  
                from: "pecas",  
                localField: "itens_peca.peca_id", // Assumindo que a chave de  
                // ligação se chama peca_id  
                foreignField: "_id",  
                as: "dados_peca"  
            }  
        },  
  
        { $unwind: "$dados_peca" },  
  
        {  
            $project: {  
                _id: 0,  
                NOME: "$dados_peca.nome",  
                QUANTIDADE: "$itens_peca.quantidade",  
                VALOR_UNITARIO_VENDA: "$itens_peca.valor_venda"  
            }  
        }  
    ]).toArray();  
}
```

```

    return resultado;
}
var pecasDaOS1 = fn_ListarPecasDaOS(1);
printjson(pecasDaOS1);

```

```

[
  {
    NOME: 'Óleo Sintético 5W30 1L',
    QUANTIDADE: 4,
    VALOR_UNITARIO_VENDA: 55
  },
  {
    NOME: 'Filtro de Óleo Lubrificante',
    QUANTIDADE: 1,
    VALOR_UNITARIO_VENDA: 35
  },
  {
    NOME: 'Pastilha de Freio Dianteira',
    QUANTIDADE: 1,
    VALOR_UNITARIO_VENDA: 180
  }
]

```

30.Verificação de Alerta de Stock Mínimo para Peça Específica.
Esta consulta avalia se uma determinada peça necessita de reposição urgente no almoxarifado, retornando um conjunto de resultados (documento projetado).

```

var IdPeca = 3;
var StockMinimo = 20;

db.pecas.aggregate([
  {
    $match: { _id: IdPeca }
  },

```

```

{
  $project: {
    _id: 0,

    "Status do Stock": {
      $cond: {
        if: { $lt: ["$estoque_atual", StockMinimo] },
        then: "ALERTA CRÍTICO",
        else: "SITUAÇÃO REGULAR"
      }
    },

    "Mensagem de Controlo": {
      $cond: {
        if: { $lt: ["$estoque_atual", StockMinimo] },
        then: { $concat: ["A peça [", "$nome", "] necessita de novas unidades no almoxarifado!"] },
        else: { $concat: ["A peça [", "$nome", "] apresenta saldo em conformidade com as regras."] }
      }
    },

    "Stock Atual": "$estoque_atual",
    "Limite Mínimo de Segurança": { $literal: StockMinimo },

    "Diferença para Compra": {
      $cond: {
        if: { $lt: ["$estoque_atual", StockMinimo] },
        then: { $subtract: [StockMinimo, "$estoque_atual"] },
        else: 0
      }
    }
  }
}

```

```
}  
}  
}  
}  
])
```

```
{  
  'Status do Stock': 'ALERTA CRÍTICO',  
  'Mensagem de Controle': 'A peça [Pastilha de Freio Dianteira] necessita de novas unidades no a  
  'Stock Atual': 15,  
  'Limite Mínimo de Segurança': 20,  
  'Diferença para Compra': 5  
}
```

4. CONCLUSÃO

4.1 Resultados Obtidos

O desenvolvimento do sistema de gerenciamento para o Centro Automotivo, consolidado nesta etapa de migração arquitetural para o paradigma Não Relacional, proporcionou uma visão prática e profunda sobre a flexibilidade das tecnologias de banco de dados modernas. A transição do modelo tabular relacional para o modelo orientado a documentos (MongoDB) evidenciou, na prática, os benefícios de desempenho gerados pela desnormalização estratégica dos dados.

Os resultados obtidos através da execução da matriz de consultas comprovaram que a incorporação de tabelas dependentes, como o registro das peças consumidas na manutenção, diretamente em vetores (arrays) embutidos no documento BSON da Ordem de Serviço eliminou a dependência excessiva de cruzamentos estruturais complexos. Além disso, a conversão bem-sucedida das antigas rotinas relacionais (como Views e

Stored Procedures) para o uso avançado do Aggregation Framework demonstrou que o banco de dados NoSQL não apenas armazena informações de forma ágil, mas possui total capacidade para gerar inteligência de negócios, relatórios analíticos de faturamento e alertas de estoque, atendendo com excelência aos requisitos do setor automotivo.

4.2 Sugestões para Melhorias Futuras

Para a evolução do sistema de gerenciamento do Auto Center e visando uma futura expansão do escopo do banco de dados no paradigma orientado a documentos (NoSQL), sugerem-se as seguintes melhorias:

- Módulo de Auditoria e Versionamento (Logs): Adotar o padrão de projeto de versionamento de documentos (Document Versioning Pattern) ou criar uma coleção dedicada de logs para registrar qual usuário realizou alterações de status nas Ordens de Serviço ou modificações financeiras, garantindo rastreabilidade e conformidade com as boas práticas de segurança da informação.
- Controle de NF-e e Faturamento Integrado: Expandir a modelagem BSON para suportar a emissão de Notas Fiscais Eletrônicas de Serviço (NFS-e) e de Produto (NF-e). Isso pode ser alcançado criando subdocumentos aninhados para o registro das formas de pagamento e parcelamentos diretamente dentro da Ordem de Serviço concluída, facilitando a extração de dados para o fluxo de caixa da oficina.

5 REFERÊNCIAS

MONGODB. MongoDB Documentation. Disponível em: <https://www.mongodb.com/docs/>. Acesso em: 20 jun. 2026.

HOWS, David; MEMBREY, Peter; PLUGGE, Eelco. Introdução ao MongoDB. São Paulo: Novatec, 2015.

ROB, Peter; CORONEL, Carlos. Sistemas de Banco de Dados: projeto, implementação e gerenciamento. São Paulo: Cengage Learning, 2011.

DATE, C. J. Introdução a Sistemas de Banco de Dados. 8ª ed. Rio de Janeiro: LTC, 2004.

HEUSER, C. A. Projeto de Banco de Dados. 6ª ed. Porto Alegre: Bookman,

2009